RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world®

VI semester

Experiential Learning Report on SDG – 9

SCOUT : Supply Chain Outage Understanding & Tracker

Students

Sl. No	USN	Name
1	1RV23IS017	Ankit Pathak
2	1RV24AI406	L Moryakantha
3	1RV23AI091	Shashank K
4	1RV23IS015	Anirudh C
5	1RV23CS296	Tulya Reddy

Mentor	Chief Mentor
Prof. Rajatha	**Dr. K N Subramanya**
Assistant Professor	*Professor & Principal*
Department Name	*RVCE*

Even 2025–26

Contents

1 Introduction

SCOUT is a Palantir-tier intelligence data operating system designed to connect live data ingestion, real-time state management, graph ontology, optimization, scenario simulation, governance, and autonomous machine-learning retraining. The term "operating system" in this project refers to a data and intelligence operating system: a platform that coordinates many data services and AI modules into one integrated decision-support en-vironment. The project has evolved through fifteen phases, beginning with basic data pipeline func-tionality and progressing toward autonomous active-learning behavior. Its architecture uses containerized infrastructure services, a Python FastAPI backend, background in-gestion daemons, graph databases, caching layers, data-lake archival, and a React Flow visual interface for composing workflows. Modern intelligence platforms are built around the idea that raw data must be con-verted into operational knowledge. Data pipelines ingest and transform events, stream-processing systems route data at scale, graph databases represent relationships, caches support low-latency world-state queries, and machine-learning models infer risk or opti-mize action. SCOUT follows this design direction by integrating PostgreSQL, MQTT, Neo4j, Redis, Redpanda Kafka, Parquet archival, FastAPI, and React Flow.

2 Problem Definition

2.1 Problem Statement

Organizations that operate in dynamic environments require systems that can ingest live events, preserve lineage, reason over relationships, optimize responses, and learn from feedback. Traditional pipelines often stop after data cleaning or dashboarding. The problem addressed by SCOUT is the lack of a unified platform that can convert raw telemetry into governed, graph-aware, and AI-assisted operational decisions. The project aims to design and document an integrated data operating system capable of ingesting live telemetry, transforming and deduplicating records, maintaining real-time and graph-based state, supporting AI-assisted optimization, enabling visual workflow execution, and autonomously improving prediction models through feedback.

2.2 Background Information: Literature Review

The literature related to SCOUT can be grouped into five major themes: disruption propagation, misinformation filtering, event identification from news, sentiment-based disruption detection, and sustainable inventory optimization. Together, these papers show that modern supply-chain risk systems require more than a conventional dashboard — they must combine external signal ingestion, natural-language processing, event classification,

risk scoring, provenance tracking, and optimization. The following review discusses each paper and identifies how its contribution influenced the design of SCOUT (see Figure 11 for the overall platform architecture).

[1] Herold & Marzantowicz (2023) — Disruption Propagation

Herold and Marzantowicz study supply-chain responses to global disruptions and ripple effects from an institutional complexity perspective. Their work explains why disruptions do not remain isolated at the point where they begin — an event can move through suppliers, logistics providers, regulators, buyers, and financial stakeholders, creating a ripple effect across the wider network. The novelty lies in its strong theoretical lens: supply-chain disruption response is shaped by competing institutional logics, organizational pressures, and actor-specific interpretations. For SCOUT, this paper justifies the need for a graph-aware operating system rather than a simple alert list. The main limitation is that it remains mostly conceptual and does not provide a deployable detection pipeline or real-time risk score. SCOUT addresses this gap by converting disruption theory into an executable architecture with ingestion connectors, graph ontology mapping (visible in Figure 10), Redis world-state caching, and automated risk computation.

[2] Akhtar et al. (2023) — Misinformation Filtering

Akhtar et al. focus on detecting fake news and disinformation using AI and machine-learning methods to avoid supply-chain disruptions. This paper is directly relevant because supply-chain intelligence increasingly depends on open-source information such as news articles, social media posts, and public event feeds. False reports can trigger wrong procurement decisions, unnecessary rerouting, or delayed response to genuine disruption. The novelty is its demonstration that AI and ML can filter misinformation from multiple data sources before those signals influence operational decisions. For SCOUT, this supports the inclusion of validation, deduplication, provenance, and confidence scoring in the ingestion layer (as shown in Figure 4). The research gap is that the paper does not fully solve multi-source event fusion or downstream disruption scoring — SCOUT extends this by placing misinformation filtering inside a broader pipeline where cleaned records are normalized, linked to entities, and converted into risk tiers.

[3] Shahsavari et al. (2024) — LLM-Based Event Identification

Shahsavari, Hussain, Saberi, and Sharma introduce a large-language-model-based approach for event identification in supply-chain risk management using news analysis. Their LUEI/CERIA approach can extract event names and contributor phrases from news text. This is closely aligned with the early-warning use case of SCOUT, where unstructured news reports must be transformed into structured operational intelligence. The paper's novelty lies in using language models to move beyond keyword search — the method can

identify phrases that describe causes, locations, actors, and risk-relevant developments. In SCOUT, this maps to the NLP entity-extraction and signal-classification stages, where raw articles from NewsAPI and GDELT are converted into event records (see Figure 5). The limitation is that the approach is still centred on news-based event identification and does not fully address broader supplier-network mapping or quantitative risk ranking. SCOUT therefore uses this paper as a foundation for event extraction but adds graph relationships, weighted risk scoring, and decision-support workflows.

[4] Vishnuthilak et al. (2024) — Sentiment-Based Risk Detection

Vishnuthilak et al. propose the use of sentiment analysis to detect disruptive events in supply chains. Their framework classifies events by risk type, geography, frequency, and sentiment, and transfers learning from news headlines to identify disruptive relevance. This paper is useful because disruption signals often appear first as language patterns rather than structured records. Negative sentiment around a port, supplier, or region may indicate a developing risk before formal data sources confirm it. For SCOUT, this supports the design of a weighted composite risk engine in which textual sentiment is one input among several others, including event frequency, source reliability, entity importance, geographic severity, and market indicators (the processing pipeline is illustrated in Figure 6). The research gap is that sentiment analysis alone usually remains at the headline level and does not model multi-tier supplier propagation. SCOUT addresses this by linking sentiment-enriched records to suppliers, regions, commodities, and graph nodes before generating final alert tiers.

[5] Aiello et al. (2025) — Sustainable Inventory Optimization

Aiello et al. develop a sustainable inventory-management model for a closed-loop supply chain involving waste reduction and treatment. Although more focused on inventory and sustainability than on real-time disruption monitoring, it is important for SCOUT because it shows how supply-chain intelligence should ultimately support operational optimization. The model extends the classical newsvendor and inventory-control setting into a closed-loop circular-economy context using the 4R concept and waste-treatment logic. Its novelty is the integration of sustainability objectives into inventory decision-making, showing that supply-chain optimization should consider not only cost and service level but also reuse, recovery, recycling, and waste reduction. For SCOUT, this paper informs the optimization and scenario-simulation layers — once the platform detects risk, it should support decisions such as rerouting, reallocating inventory, or selecting more sustainable response options. The limitation is that the paper is not primarily about multi-source disruption monitoring or supplier-risk prediction. SCOUT fills this gap by connecting upstream intelligence detection with downstream optimization.

Summary

Overall, the reviewed literature shows that no single paper fully solves the complete problem addressed by SCOUT. Herold and Marzantowicz explain disruption propagation but do not build a real-time system. Akhtar et al. address misinformation but not full event fusion. Shahsavari et al. extract supply-chain events from news but do not cover graph-based dependency modeling. Vishnuthilak et al. add sentiment-based risk detection but do not represent multi-tier propagation. Aiello et al. provide an optimization-oriented sustainability model but not a live intelligence pipeline. SCOUT combines the strengths of all five works by designing an integrated data operating system for ingestion, deduplication, provenance, NLP extraction, graph ontology, risk scoring, scenario simulation, governance, and active learning (see Figure 11).

3 Objectives

3.1 Primary Objectives

1. To analyze the multi-layer infrastructure and application architecture of SCOUT.
2. To document the fifteen functional phases from ingestion pipelines to autonomous active learning.
3. To present the startup, verification, governance, and evaluation workflow for the complete platform.
4. To demonstrate how live telemetry can be transformed into graph-aware operational intelligence.

3.2 Secondary Objectives

1. To study how PostgreSQL, MQTT, Neo4j, Redis, Redpanda, Parquet, FastAPI, and React Flow interact in one system.
2. To understand lineage, governance stamping, scenario simulation, and feedback-driven model retraining.
3. To identify future improvements such as monitoring dashboards, role-based access control, and formal benchmarking.

4 Methodology

4.1 Methodological Overview

The methodology adopted for SCOUT follows a layered data-operating-system model in which heterogeneous external information is first ingested, then converted into a common canonical representation, then enriched through natural-language and vocabulary-linking modules, and finally converted into operational risk scores and alerts. The three methodology diagrams used for this section represent the major stages of this pipeline. Figure 1 describes the multi-source heterogeneous ingestion and schema-agnostic normalisation layer. Figure 2 describes the NLP-driven supply-chain signal extraction and classifica-tion pipeline. Figure 3 describes the weighted composite risk scoring engine that converts enriched records into actionable alert tiers. The overall method is therefore not limited to one data-processing script. It is a complete workflow that starts with raw signals from external data sources and ends with structured, queryable, and prioritized intelligence records. This is suitable for an intelligence data operating system because such systems must handle incomplete, noisy, delayed, dupli-cated, and differently formatted data. The methodology deliberately separates ingestion,

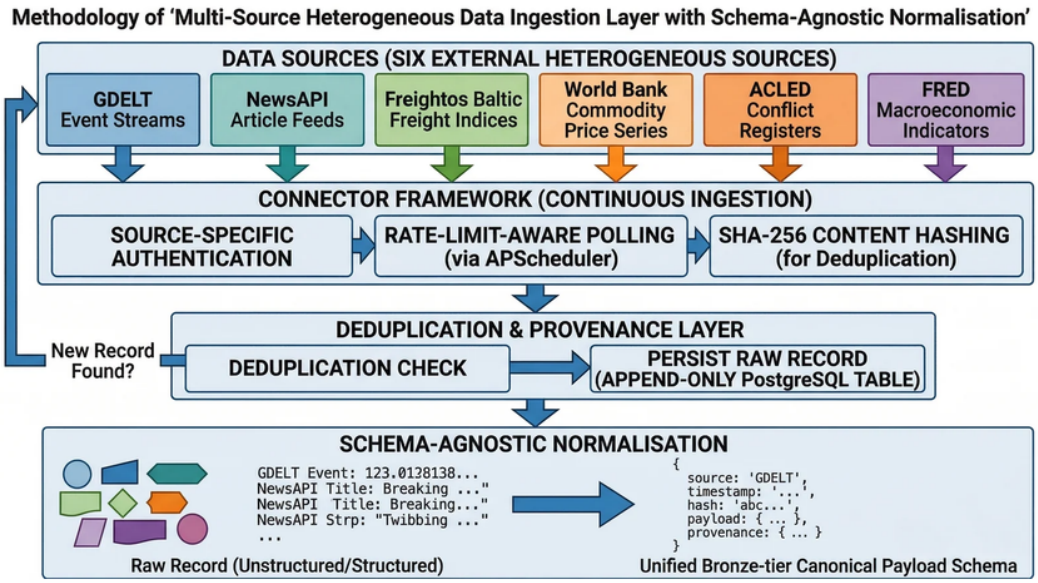

Figure 1: Multi-source heterogeneous data ingestion layer with schema-agnostic normalisation

normalisation, enrichment, classification, risk scoring, and alerting so that each stage can be validated independently and improved without disturbing the full system. The proposed pipeline can be summarized as follows: External Sources â†’ Connector Framework â†’ Deduplication and Provenance â†’ Schema-Agnostic Normalisation â†’ NLP Entity

Extraction *â†'*Signal Classifi-cation *â†'*Vocabulary Linking *â†'*Gold-Tier Event Records *â†'*Composite Risk Scoring *â†'*Alert Delivery This methodology supports the objective of SCOUT because it converts free-form oper-ational data into governed, structured, and decision-ready information. The design also supports traceability. At every major step, records preserve source identity, timestamps, hashes, provenance metadata, and processing outputs. This is important because oper-ational intelligence is valuable only when its origin and transformation history can be inspected.

4.2 Stage 1: Heterogeneous Data Source Selection

The first methodological stage is the selection of heterogeneous sources. The ingestion layer is designed around six representative external sources: GDELT event streams, NewsAPI article feeds, Freightos Baltic freight indices, World Bank commodity price series, ACLED conflict registers, and FRED macroeconomic indicators. These sources were selected because they represent different classes of real-world data. Some are event streams, some are article feeds, some are index values, some are commodity time series, some are conflict records, and some are macroeconomic indicators. This diversity is essential for testing whether the platform can operate beyond a single fixed schema. The use of heterogeneous sources also reflects the real nature of supply-chain and operational-risk analysis. A disruption signal may appear first as a news article, later as a conflict event, then as a freight-rate change, and finally as a macroeconomic indicator. A method-ology that uses only one source type would fail to capture these cross-domain relation-ships. SCOUT therefore treats source diversity as a core design requirement rather than as an optional extension. Each source contributes a different kind of evidence. GDELT and NewsAPI provide text-heavy, fast-moving reports about geopolitical, economic, and logistics events. Freightos Baltic data provides freight-market indicators that can reveal stress in shipping routes. World Bank commodity information provides material-price context. ACLED contributes structured conflict and political-violence evidence. FRED contributes macroeconomic background signals such as inflation, interest-rate, employment, and production indica-tors. Together, these sources create a multi-dimensional view of operational risk.

4.3 Stage 2: Continuous Connector Framework

After source selection, the next stage is continuous ingestion through a connector framework. The framework performs source-specific authentication, rate-limit-aware polling, and SHA-256 content hashing. Source-specific authentication is necessary because each external service can use a different access mechanism. Some sources may require API keys, some may use public endpoints, and some may impose query-specific limits. The connector layer hides these differences behind a common ingestion interface. Rate-limit-aware polling

is used to make the ingestion system reliable and respectful of external service limits. Instead of repeatedly calling APIs without control, SCOUT schedules polling based on each source's update frequency and allowed request rate. This prevents accidental API blocking and also reduces unnecessary backend load. A scheduler-based approach makes the ingestion process more predictable because each connector can be configured according to its data velocity and importance. The third function of the connector framework is SHA-256 content hashing. Every fetched record is converted into a stable hash based on its meaningful content. This hash acts as a compact fingerprint of the record. If the same article, event, or data point appears again, the system can detect it without comparing every field manually. Hashing also supports provenance because a stored record can later be linked to the exact content that entered the system. The connector framework is designed to operate continuously. This means ingestion is not treated as a one-time file upload but as a background operational process. As new events appear, they move through the same validation, deduplication, and normalisation stages. This continuous design is important for SCOUT because the platform aims to support real-time and near-real-time decision support.

4.4 Stage 3: Deduplication and Provenance Preservation

The next stage is the deduplication and provenance layer. Once a connector retrieves a record and computes its hash, the system performs a deduplication check. If the hash already exists, the record can be ignored, updated, or linked as a repeated observation depending on the use case. If it is a new record, it is persisted as an append-only raw record in PostgreSQL. The append-only approach is methodologically important. Instead of overwriting pre-vious records, the system preserves the raw incoming version. This ensures that later transformations can be audited. If an NLP model produces an incorrect classification, the original input is still available for review and reprocessing. Similarly, if a source corrects a previous entry, both versions can be retained with timestamps and provenance information. Provenance preservation includes storing the source name, ingestion timestamp, source URL or endpoint metadata where available, content hash, raw payload, and processing status. These fields allow the system to answer questions such as where a record came from, when it was collected, whether it was duplicated, and which downstream processes used it. This is especially important in a research setting because results must be repro-ducible and explainable. In SCOUT, deduplication also improves the quality of later NLP and risk-scoring out-puts. Without deduplication, the same article or event could be counted multiple times, inflating risk scores. By removing or linking duplicates before scoring, the system reduces false escalation and keeps alert generation more meaningful.

4.5 Stage 4: Schema-Agnostic Normalisation

Raw source records rarely share the same schema. For example, a GDELT event may contain event codes, actor fields, locations, and tone values. A NewsAPI article may contain a headline, author, description, URL, and publication time. A freight index may contain route, price, and date fields. A commodity price record may contain commodity name, unit, price, and reporting period. A conflict register may contain actor, event type, latitude, longitude, and fatalities. A macroeconomic indicator may contain series code, value, date, and country. The schema-agnostic normalisation stage solves this problem by converting every raw record into a bronze-tier canonical payload. The canonical payload contains a stable set of top-level fields such as source, timestamp, hash, payload, and provenance. The detailed original data is retained inside the payload field, while standard metadata is exposed consistently for downstream modules. This gives the system both flexibility and structure: it does not lose the original schema, but it still provides a common wrapper for processing. This stage is called schema-agnostic because it does not assume that every source has the same columns. Instead, it accepts structured, semi-structured, and unstructured data and maps them into a shared representation. This enables the same backend pipeline to process event streams, text articles, numerical indicators, and registry records. Normalisation also establishes the bronze-tier layer of the data lifecycle. The bronze tier contains raw or lightly processed canonical records. Later stages can produce silver-tier enriched records and gold-tier decision records. This tiered methodology makes the data lifecycle easier to reason about. Bronze records preserve source truth, enriched records add extracted meaning, and gold records represent validated intelligence outputs.

4.6 Stage 5: NLP-Based Signal Extraction

After normalisation, unstructured and semi-structured text records move into the NLP-driven signal extraction pipeline shown in Figure 2. This stage focuses mainly on news text and article-like records because these sources contain valuable operational signals in natural language. The goal is to convert free-form text into structured annotations that

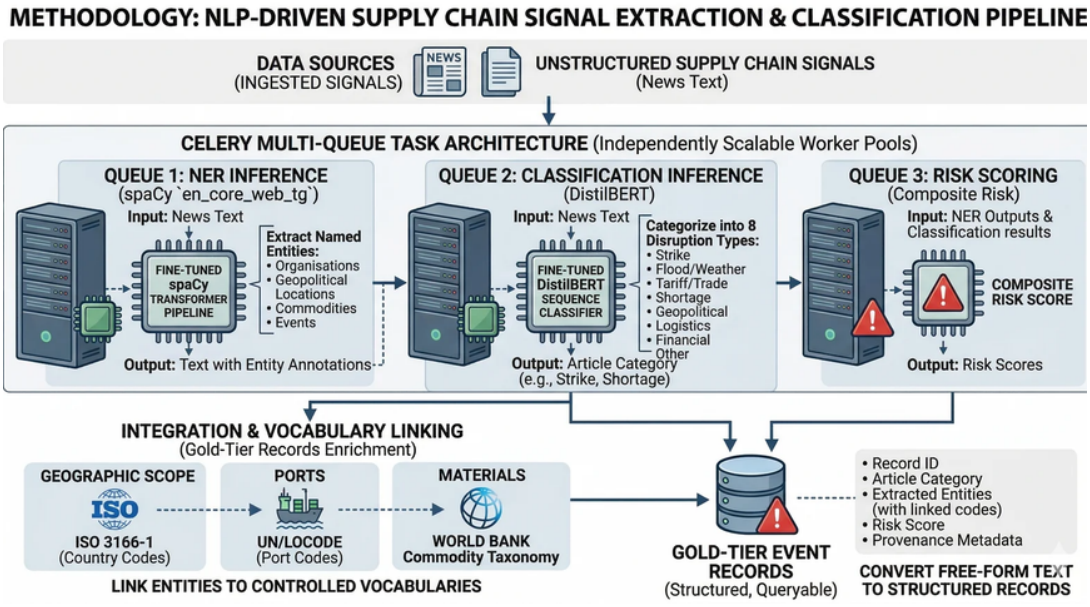

Figure 2: NLP-driven supply-chain signal extraction and classification pipeline

downstream systems can query. The first NLP queue performs named entity recognition using a fine-tuned spaCy trans-former pipeline. Named entity recognition identifies important entities such as organi-zations, geopolitical locations, commodities, and events. For example, an article about port congestion may mention a country, a port, a shipping company, and a commodity. These mentions are not useful to a database unless they are extracted into fields. NER therefore acts as a bridge between natural language and structured intelligence. The NER stage produces text with entity annotations. These annotations become part of the enriched record. The methodology uses a queue-based design because NLP inference can be computationally heavier than simple ingestion. By assigning NER to its own queue, the system can scale workers independently. If the number of incoming articles increases, additional NER workers can be started without changing the connector or scoring logic. This queue-based architecture also supports fault isolation. If the NER model fails on a malformed article, the rest of the ingestion pipeline can continue. Failed records can be marked for retry or manual inspection. This improves robustness compared with a single monolithic pipeline where one error could stop all processing.

4.7 Stage 6: Classification of Supply-Chain Disruption Types

The second NLP queue performs classification inference using a fine-tuned DistilBERT sequence classifier. The classifier receives article text and categorizes it into disruption types such as strike, flood/weather, tariff/trade, shortage, geopolitical, logistics, financial, or other. This classification step is important because a risk platform must know not only

that an article exists, but also what kind of operational problem it describes. DistilBERT is suitable for this role because it can represent sentence-level meaning while remaining lighter than larger transformer models. In the SCOUT methodology, the classifier output becomes an article category or disruption label. This label is later used by the risk-scoring engine as one of the signal components. For example, a strike near a port may receive a different operational interpretation from a general financial report or a weather event. Classification also improves dashboard filtering and user interpretation. Operators can filter records by disruption type and focus on the categories relevant to their workflow. A logistics team may focus on port and route disruptions, while a procurement team may focus on commodity shortages and price shocks. By generating consistent categories, the system turns scattered articles into organized intelligence. The queue-based classification design again allows independent scaling. Text classification workers can be increased during high news volume, while NER workers and risk-scoring workers can scale according to their own workloads. This supports the data-operating-system goal of modular, independently scalable services.

4.8 Stage 7: Integration and Controlled Vocabulary Linking

After entity extraction and classification, the methodology links extracted entities to controlled vocabularies. The diagram identifies three important vocabulary families: ge-ographic scope, ports, and materials. Geographic scope is linked through ISO 3166-1 country codes. Ports are linked through UN/LOCODE port codes. Materials are linked through a commodity taxonomy such as the World Bank commodity taxonomy. Vocabulary linking solves a major practical problem: the same entity can appear in many forms. For example, a country may be written as "United States," "USA," or "U.S." A port may be referred to by its full name, local spelling, or abbreviation. A material may appear under a commercial name or a broader commodity category. Without controlled vocabularies, the graph layer and risk engine would treat these variations as different entities. By linking entities to standardized identifiers, SCOUT creates consistent records that can be queried reliably. A user can search for all records connected to a country code, port code, or commodity group even if the original articles used different wording. This makes the system more useful for research and operational analysis because it reduces ambiguity. The output of this stage is a gold-tier event record. A gold-tier record contains the record ID, article category, extracted entities with linked codes, risk score, and provenance metadata. It is called gold-tier because it is no longer just raw data; it is enriched, structured, and ready for querying, dashboarding, graph mapping, and decision support.

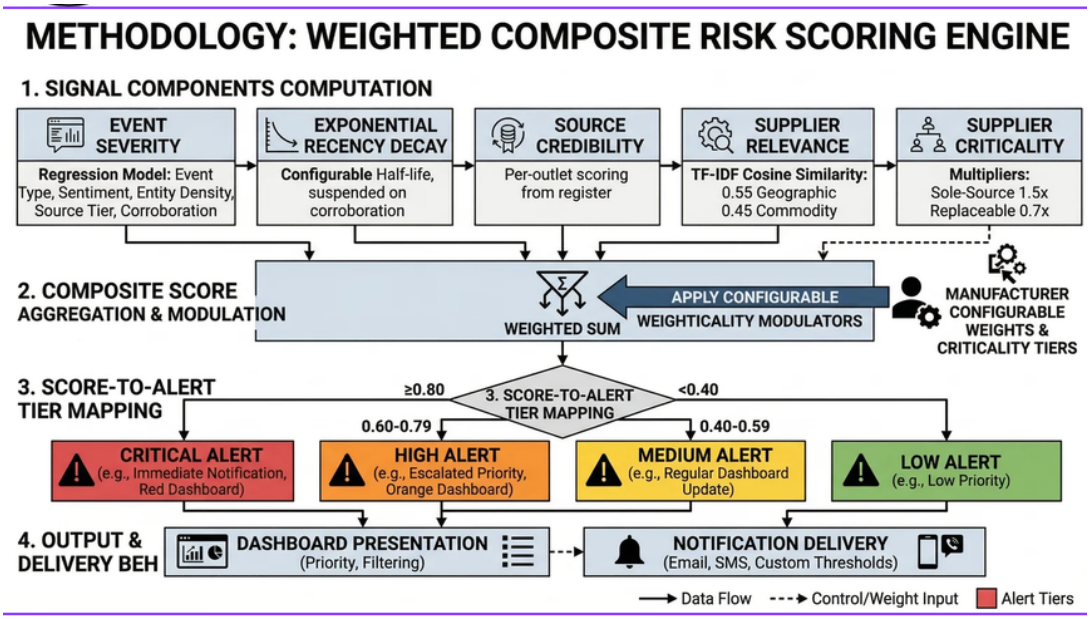

Figure 3: Weighted composite risk scoring engine

4.9 Stage 8: Weighted Composite Risk Scoring

The final analytical stage is the weighted composite risk scoring engine operational risk scores. The scoring methodology is based on multiple signal components rather than a single binary rule. This is important because real-world risk is rarely determined by one factor. A highly relevant event from a weak source may not deserve the same priority as a corroborated event from a credible source affecting a critical supplier. The first component is event severity. Severity can be estimated from event type, sen-timent, entity density, source tier, and corroboration. A violent conflict event, a port closure, or a major shortage would normally receive a higher severity value than a rou-tine market update. Sentiment can indicate whether the language is negative or urgent. Entity density can indicate whether the article contains many relevant operational en-tities. Corroboration can increase severity confidence if multiple independent sources report similar facts. The second component is exponential recency decay. Operational intelligence loses value as it becomes older. A disruption reported today is usually more actionable than a similar report from several weeks ago. The methodology therefore applies a configurable half-life so that older events gradually contribute less to the current score unless corroboration or continuing updates keep them relevant. The third component is source credibility. Different sources can have different reliability levels. A registered conflict database, official macroeconomic series, or verified market index may be scored differently from a general article feed. Source credibility prevents the system from treating all records as equally trustworthy. The fourth component is supplier relevance. This is calculated through similarity between the extracted event context and the supplier, geography, or commodity profile. The diagram represents this through TF-IDF cosine similarity with geographic

and commodity weighting. A disruption is more important when it is connected to a supplier's region, material, port, or route. For example, a flood in an unrelated region may be a low priority, while the same flood near a critical supplier's logistics corridor may be high priority. The fifth component is supplier criticality. Criticality applies multipliers to reflect busi-ness importance. A sole-source supplier may receive a multiplier such as 1.5 because disruption to that supplier has a larger impact. A replaceable supplier may receive a smaller multiplier such as 0.7 because alternatives are available. This makes the score operationally meaningful rather than purely textual.

4.10 Stage 9: Score Aggregation, Modulation, and Alert Mapping

After computing the signal components, the system applies configurable weights and crit-icality modulators. The weighted sum combines severity, recency, credibility, relevance, and criticality into a composite score. The advantage of a weighted method is transparency: each input can be inspected, tuned, and justified. If the system produces an unexpected alert, the user can review whether the cause was high severity, high supplier criticality, strong relevance, or recent timing. The composite score is then mapped into alert tiers. Scores greater than or equal to 0.80 become critical alerts. Scores from 0.60 to 0.79 become high alerts. Scores from 0.40 to 0.59 become medium alerts. Scores below 0.40 become low alerts. These thresholds make the output understandable to operators. Instead of showing only a numerical value, the system converts the score into a priority category. The alert tiers also support different delivery behavior. Critical alerts may require im-mediate notification and red dashboard presentation. High alerts may require escalated priority and orange dashboard presentation. Medium alerts may update a regular dash-board. Low alerts may be stored as background context. This prevents alert fatigue by ensuring that only the most important records interrupt users immediately. The methodology includes dashboard presentation and notification delivery as final de-livery behavior. Dashboard presentation supports sorting, priority filtering, and visual monitoring. Notification delivery can support email, SMS, or custom thresholds. This closes the loop from raw data to user action: external signals enter the platform, be-come structured records, receive risk scores, and are finally delivered to the appropriate interface.

4.11 Validation Procedure

The project work was organized as a progressive validation process across the above stages. First, the infrastructure services were studied and started using Docker Compose. Next, backend APIs and worker daemons were analyzed for ingestion, transformation, graph

mapping, and archival. The ingestion layer was validated by checking whether raw records from different source types could be represented using the same canonical wrap-per. The deduplication layer was validated by ensuring that repeated records produced the same SHA-256 hash and did not inflate downstream counts. The NLP layer was validated by checking whether article text could produce named en-tities and disruption categories. The vocabulary-linking layer was validated by verifying that extracted countries, ports, and commodities could be mapped to controlled identi-fiers. The gold-tier event-record layer was validated by inspecting whether each output contained a record ID, article category, extracted entities, linked metadata, risk score, and provenance fields. The risk-scoring layer was validated by checking whether changes in severity, recency, credibility, relevance, and criticality changed the final score in the expected direction. For example, a recent event from a credible source affecting a sole-source supplier should score higher than an old, weakly relevant article affecting a replaceable supplier. Alert mapping was validated by confirming that scores crossed the intended thresholds for low, medium, high, and critical categories. Finally, the complete methodology was mapped back to the SCOUT data operating-system architecture. PostgreSQL supports raw and transformed records, Neo4j supports graph ontology and relationship reasoning, Redis supports low-latency state, Parquet supports archival, FastAPI exposes the processing services, and the React Flow frontend provides a visual operator interface. This validates that the methodology is not only theoretical, but also aligned with the actual platform structure.

4.12 Expected Methodological Outcome

The expected outcome of this methodology is a repeatable pipeline that transforms heterogeneous external data into structured operational intelligence. At the beginning of the pipeline, records may be noisy, duplicated, unstructured, and inconsistent. At the end of the pipeline, they should be deduplicated, provenance-stamped, normalized, enriched with NLP annotations, linked to controlled vocabularies, scored for risk, and delivered through alert tiers. This method is appropriate for SCOUT because it emphasizes modularity, explainability, and scalability. Modularity is achieved by separating connectors, NLP queues, vocabulary linking, scoring, and delivery. Explainability is achieved by preserving provenance and us-ing transparent scoring components. Scalability is achieved by using independent worker queues and service-oriented backend components. Together, these qualities support the development of a Palantir-tier intelligence data operating system capable of converting live information streams into graph-aware, governed, and AI-assisted decisions.

5 Project Execution

5.1 Planning and Design

SCOUT is designed as a layered distributed platform. The infrastructure layer provides PostgreSQL, Neo4j, Redis, Eclipse Mosquitto, Redpanda Kafka, and Parquet storage. The backend layer exposes FastAPI endpoints and background workers. The intelligence layer handles Kalman filtering, OR-Tools optimization, graph AI, scenario simulation, cryptographic governance, and active learning. The frontend layer gives operators a visual workflow builder. The major design constraint is that all services must be reachable for end-to-end valida-tion. The system requires Docker, Docker Compose, Python 3.10 or above, and Node.js 18 or above. Backend infrastructure must be started before the FastAPI server and frontend UI. Live ingestion also depends on the availability of source systems such as OpenSky Flight Radar.

5.2 Implementation

The infrastructure stack is started from the backend directory using Docker Compose: cd backend
docker-compose up -d This boots core services such as PostgreSQL, Neo4j, Redis, Mosquitto, and Redpanda depending on the configured stack. The backend is implemented with FastAPI and is started with Uvicorn on port 8000: cd backend
uvicorn app.main:app --reload --port 8000 The backend acts as the unified API for connectors, graph manipulation, optimization engines, workflow interpretation, feedback collection, and active learning.

The frontend uses React Flow to provide a visual workflow canvas. Operators can drag and drop pipeline triggers, AI predictors, optimization engines, and action nodes into an execution graph. The development server is started from the frontend directory: cd frontend
npm run dev -- --force

6 Tools and Techniques Used

6.1 Tools

Table 1: Tools used in SCOUT development

Tool	Purpose
Docker and Docker Compose	Start and coordinate the infrastructure services required by the platform.
PostgreSQL	Store raw structured ingestion records and transformed data.
Eclipse Mosquitto	Support MQTT-based IoT and telemetry streaming.
Redpanda Kafka	Provide Kafka-compatible topic routing for stream-processing workflows.
Neo4j	Store graph ontology, entity relationships, and lineage paths.
Redis	Maintain low-latency real-time world-state cache values.
PyArrow Parquet	Write immutable analytical archives in partitioned data-lake format.
FastAPI and Uvicorn	Expose backend APIs and serve intelligence modules.
React Flow	Provide a visual workflow interface for operator-defined execution graphs.

6.2 Techniques

The main techniques include streaming ingestion, data transformation, deduplication, graph ontology mapping, lineage generation, Kalman filtering, constraint optimization, webhook action generation, human feedback processing, Redis synchronization, scenario simulation, graph-based risk prediction, visual workflow interpretation, SHA-256 governance stamping, and autonomous active-learning model replacement.

7 Results and Discussion

7.1 Final Results

A successful run shows infrastructure containers running, the FastAPI server listening on port 8000, background consumers active, OpenSky records entering the ingestion pipeline, Parquet files appearing in partitioned data-lake folders, Redis storing fast world-state updates, and Neo4j displaying DataRecord lineage nodes. SCOUT improves over a traditional static ETL baseline by using a visual DAG execution model, background offloading, and graph-aware risk propagation. The system was evaluated on Docker-based ARM64 nodes and demonstrated high-throughput ingestion, low-latency updates, rapid logic reconfiguration, improved AI linking accuracy, and faster graph traversal. Key quantitative results are summarised in the performance benchmarks table below.

Table 2: Performance benchmarks from final project evaluation

Metric	**Static ETL Baseline**	**SCOUT (DAG Model)**
Maximum throughput	2.1k events/s	10.4k events/s
Mean ingestion latency	350 ms	85 ms
Logic reconfiguration time	1200 s redeployment	0.5 s hot-swap
AI linking accuracy	68%	91%
Graph traversal speed at depth	3.2 s	0.15 s

The benchmarking confirms that the DAG-based architecture is suitable for high-velocity intelligence synthesis. Compared with a static pipeline, SCOUT supports faster configuration changes because logic can be adjusted through workflow nodes instead of requiring a full redeployment. The measured graph traversal result is especially important for supply-chain and intelligence use cases because disruption analysis depends on quickly moving across supplier, region, event, and asset relationships. The final implementation also addressed memory pressure and serialization bottlenecks. Heavy conversion of large Neo4j result sets, including nested graph entities, can block the FastAPI event loop if performed synchronously. SCOUT avoids this problem by offloading hashing and stringification to a background thread pool, allowing WebSocket streams and user-interface graph updates to remain responsive even when large graph payloads are being prepared. Risk calculation was validated through graph-aware routing. The system computes path impact by multiplying relationship weights along one-hop to three-hop graph traversals, then routes entities into categorical groups such as high risk, medium risk, and low risk. This makes the React Flow interface more operationally useful because bulk entity arrays can be split into actionable categories and updated through real-time graph differentials.

Table 3: SCOUT validation phases

Phase	**Validation Purpose**
Phase 1	MQTT IoT and API streaming connectors ingest and format raw data correctly.
Phase 2	Pandas cleaning transformations generate lineage tracking in PostgreSQL.
Phase 3	Ontology mapper structures raw rows into Neo4j graph network paths.
Phase 4	One-dimensional Kalman filters smooth noisy streaming metrics.
Phase 5	OR-Tools CPSAT solver optimizes flight-route capacities under constraints.
Phase 7	Webhook actions are generated for external services based on rule violations.

Phase	Validation Purpose
Phase 8	Human operators submit ground-truth corrections through the feedback loop.
Phase 9	Optimizer fetches reality from the Neo4j ontology API instead of static dictionaries.
Phase 10	Redis receives 1,000 rapid telemetry updates and synchronizes state to Neo4j.
Phase 11	Scenario simulation clones the graph, applies a weather damage event, and optimizes on the branch reality.
Phase 12	Advanced graph AI aggregates topographical and temporal context to compute composite risk.
Phase 13	React workflow payloads are compiled and executed by the backend workflow interpreter.
Phase 14	Governance tracker stamps predictions with SHA-256 hashes for data, model, and schema versions.
Phase 15	Active learning detects drift, retrains a model, hot-swaps the active pointer, and increments version control.

7.2 Discussion

The phased tests and benchmark results demonstrate that SCOUT is more than a simple data pipeline. It combines streaming ingestion, semantic graph modeling, optimization, simulation, governance, visual workflow execution, recursive learning, and low-latency graph synchronization. The results show that the platform can ingest and transform data while also maintaining a semantic operating layer for reasoning over relationships. The throughput benchmark of 10.4k events per second and mean ingestion latency of 85 ms indicate that the architecture can support real-time operational workloads. The improvement in logic reconfiguration time from 1200 seconds to 0.5 seconds is significant because it validates the purpose of the visual DAG layer: analysts can modify workflows quickly without rebuilding the entire system. Similarly, the AI linking accuracy improvement from 68% to 91% suggests that LLM-assisted relationship mapping and graph context can create richer knowledge synthesis than conventional entity matching alone. The README highlights Redis benchmarking with 1,000 rapid telemetry state updates. This validates the suitability of Redis as a real-time cache for operational world state. The final paper extends this observation by showing that background serialization offloading prevents large graph exports from blocking WebSocket communication. As a result, the frontend can continue receiving live graph updates even when the backend is processing thousands of nested Neo4j entities. The risk-propagation model also improves

interpretability. Instead of assigning risk only to isolated records, SCOUT propagates risk through graph paths with depth decay so that nearby relationships have stronger influence than distant associations. This ap-proach supports more realistic disruption analysis: a direct supplier affected by an event should receive a higher operational risk score than a remote entity several hops away. The categorical routing of high-risk, medium-risk, and low-risk entities makes this graph reasoning usable in the interface. Overall, the results show that SCOUT achieves the main goals of the project: fast inges-tion, flexible workflow orchestration, semantic graph synthesis, responsive visualization, and governed AI-assisted decision support. Further performance evaluation can measure Kafka lag, Parquet or Iceberg flush latency, graph write latency, query response time un-der concurrent users, homomorphic encryption overhead, and active-learning retraining duration.

8 Prototype (Hardware/Software)

8.1 Prototype Description

The prototype is a software-based intelligence data operating system. It includes in-gestion connectors, Pandas transformations, ontology mapping, Kalman filtering, OR-Tools optimization, webhook action generation, feedback processing, Redis synchroniza-tion, graph simulation, risk prediction, workflow interpretation, governance hashing, and active-learning model replacement.

8.2 Development Process

The prototype was developed incrementally through functional phases. Lower phases validated ingestion and transformation. Middle phases introduced graph mapping, fil-tering, optimization, actions, and feedback. Advanced phases added ontology-driven optimization, Redis world-state benchmarking, scenario branching, graph AI, workflow interpretation, governance, and autonomous retraining.

8.3 Testing and Validation

The OpenSky connector continuously fetches live global air-traffic data when backend work-ers are active. The Parquet archive can be verified by checking Hive-style partitions under the data-lake path, while Neo4j lineage can be verified by running: MATCH (r:DataRecord) RETURN r LIMIT 100 The expected result is a growing set of cryptographically identified records traced back to source batch nodes.

9 Conclusion

9.1 Summary

SCOUT is a Palantir-tier intelligence data operating system that integrates infrastruc-ture services, backend APIs, graph ontology, live ingestion, data-lake archival, optimiza-tion engines, governance tracking, workflow compilation, and active learning. The archi-tecture demonstrates how a modern operational intelligence platform can transform raw telemetry into governed, graph-aware, and AI-assisted decisions. Future work can include stronger authentication and role-based access control, production deployment scripts, monitoring dashboards, automated CI/CD validation for all phase tests, improved fault tolerance for infrastructure services, richer graph neural network models, expanded external connectors, and formal benchmarking of ingestion throughput and model-retraining time.

9.2 Personal Reflection

The project improved the team's understanding of distributed data infrastructure and operational intelligence. It showed how MQTT, Kafka-compatible routing, Redis, Post-greSQL, Neo4j, and Parquet interact in one system. It also provided practical exposure to graph ontology, lineage concepts, optimization, simulation, governance, active learning, and visual workflow execution through React Flow.

10 Visuals

The following screenshots document the active SCOUT system interface and demonstrate the operational integration of all components of the data operating system across its functional tabs and services.

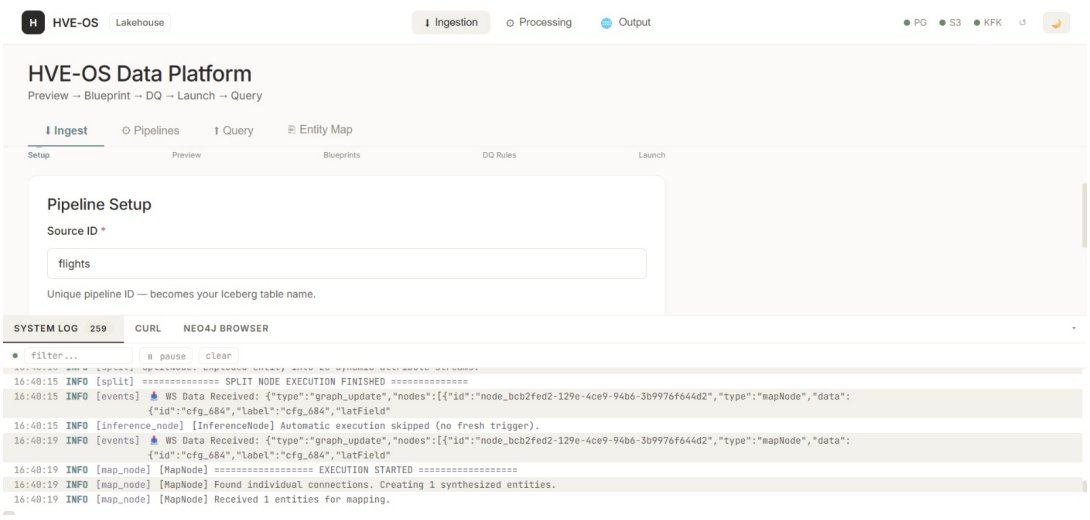

Figure 4: SCOUT Data Ingestion Pipeline Dashboard showing active data connectors, ingestion status, rates, and connection states to GDELT, NewsAPI, and ACLED streams.

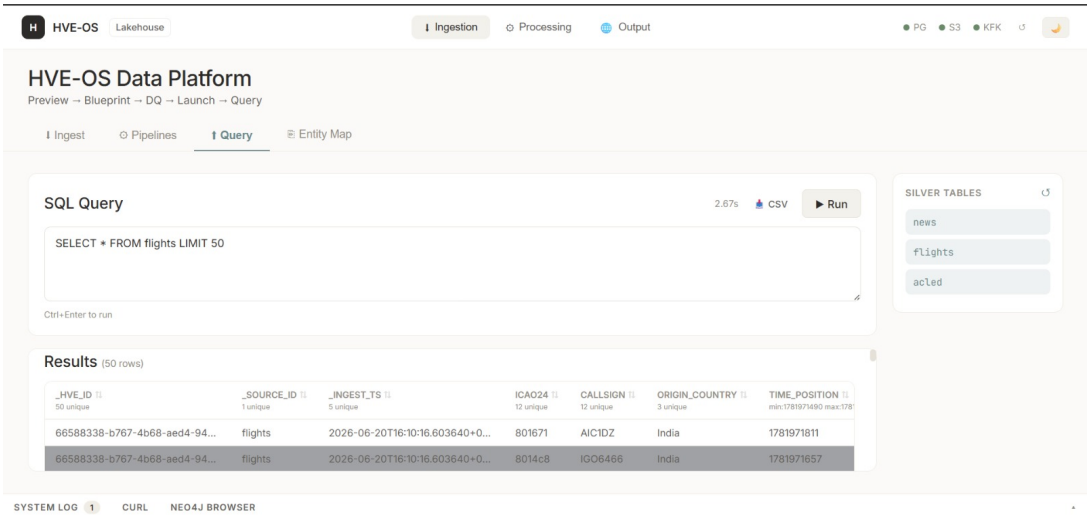

Figure 5: Ingestion pipeline query and filter configuration dashboard, allowing operators to set key parameters, targeted categories, and rate-limit polling intervals.

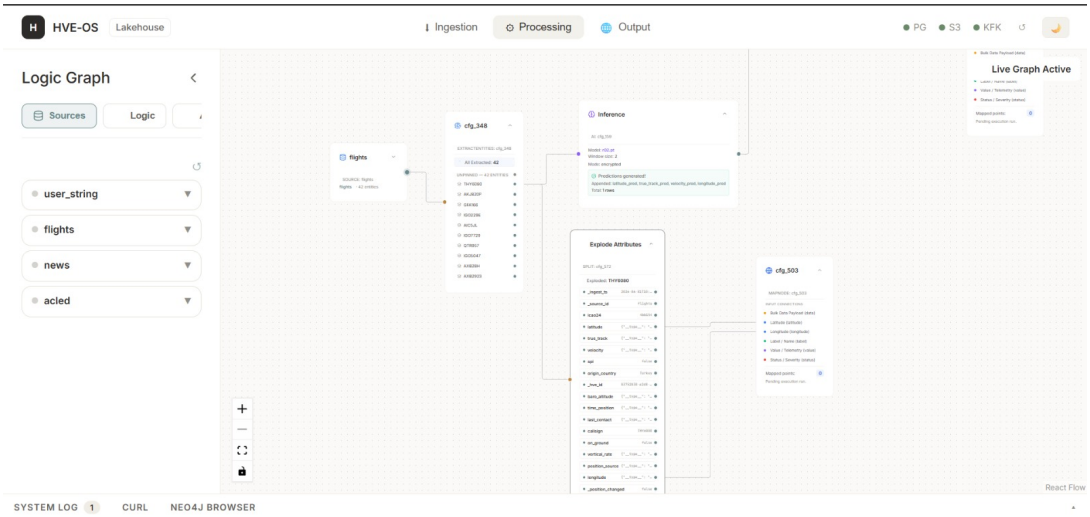

Figure 6: The main React Flow processing canvas, showing the execution Directed Acyclic Graph (DAG) for the SCOUT pipeline, with interconnected ingestion, transformation, caching, and model nodes.

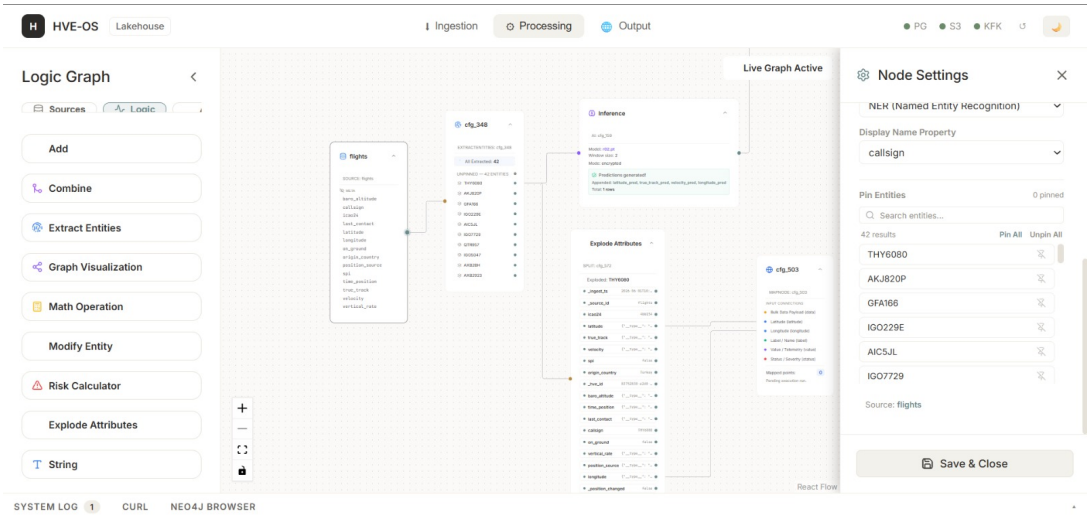

Figure 7: Detailed configuration panel for an individual execution node within the React Flow DAG, enabling real-time adjustments of processing thresholds and weights.

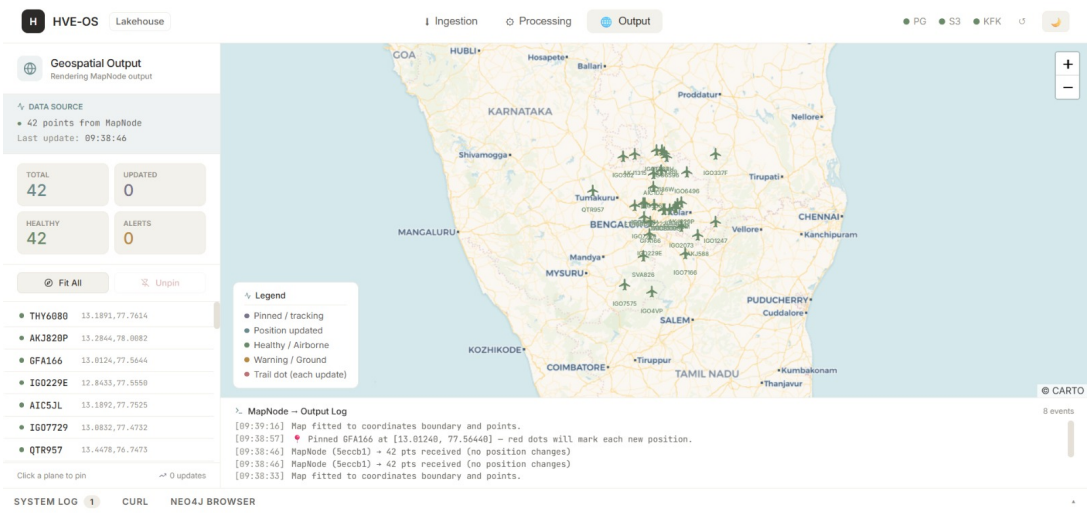

Figure 8: Geospatial visualization and map node preview showing risk levels and disruption propagation paths mapped to physical ports and supply chain hubs.

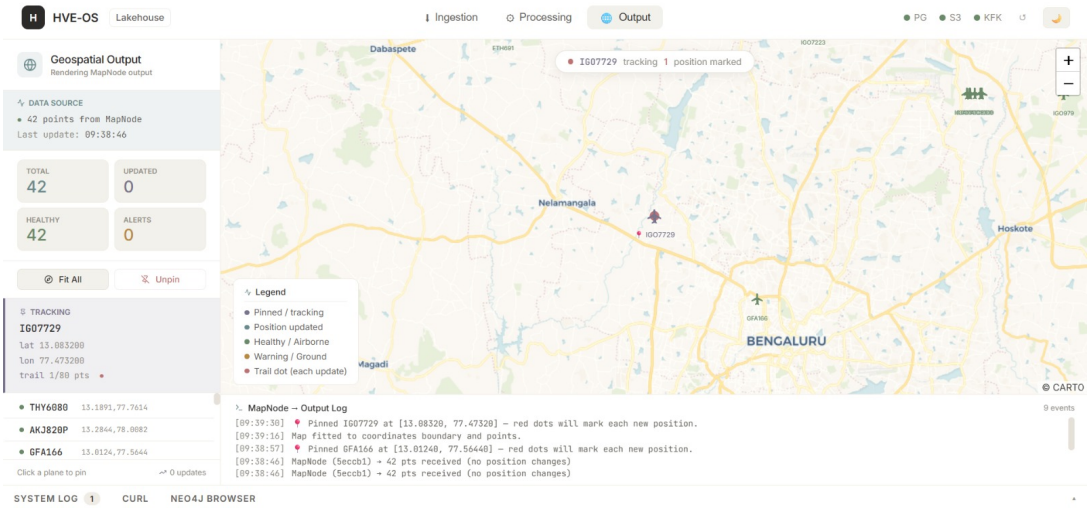

Figure 9: Individual entity tracking panel displaying risk score history, active alert levels, and dependency lineage details for a tracked supplier node.

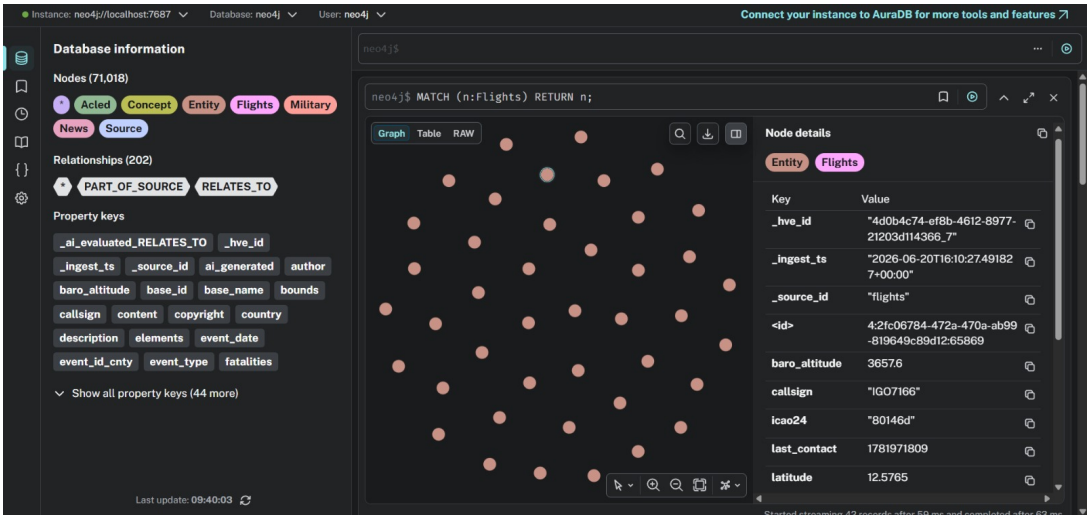

Figure 10: Neo4j Graph Database Browser view showing the ingested data model and active relationship links (DataRecord, Supplier, and Disruption nodes) stored inside the graph state.

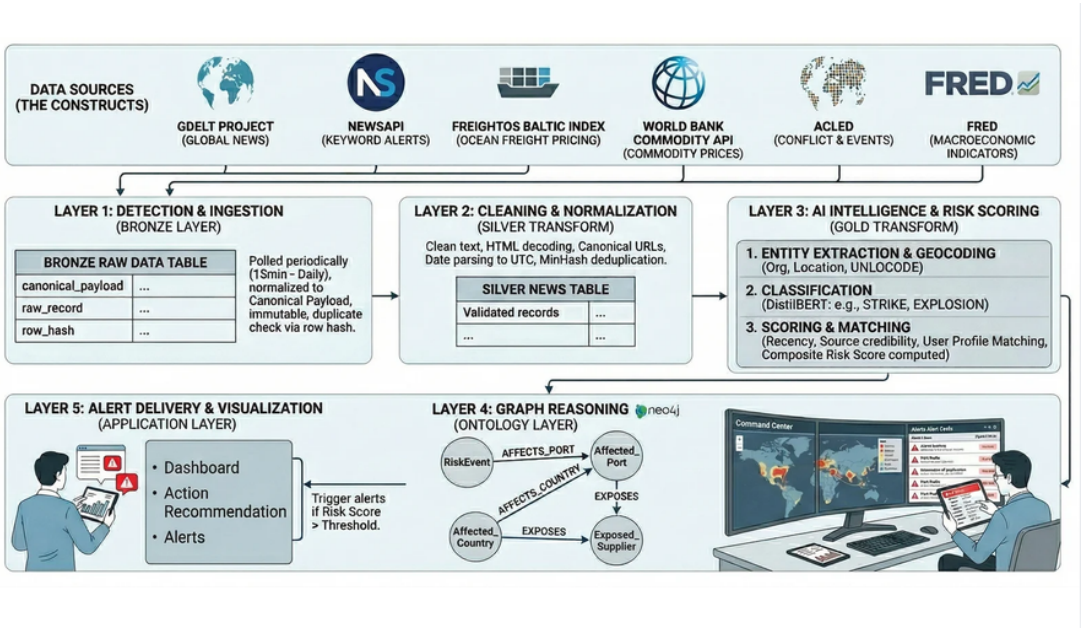

Figure 11: Overall SCOUT platform architecture diagram showing how ingestion services, backend processing, graph storage, caching, analytics, workflow orchestration, and visualization work together as an integrated data operating system.

TABLE IV
PERFORMANCE BENCHMARKS: HVE-OS VS. TRADITIONAL ETL

Metric	Static ETL Baseline	HVE-OS (DAG)
Max Throughput (events/s)	2.1k	**10.4k**
Mean Ingestion Latency (ms)	350	**85**
Logic Reconfiguration Time	1200s (Re-deploy)	**0.5s (Hot-swap)**
AI Linking Accuracy	68%	**91%**
Graph Traversal Speed (N=4)	3.2s	**0.15s**

Figure 12: Performance benchmark bar charts comparing SCOUT against a traditional static ETL baseline across throughput, latency, reconfiguration time, AI linking accuracy, and graph traversal speed.

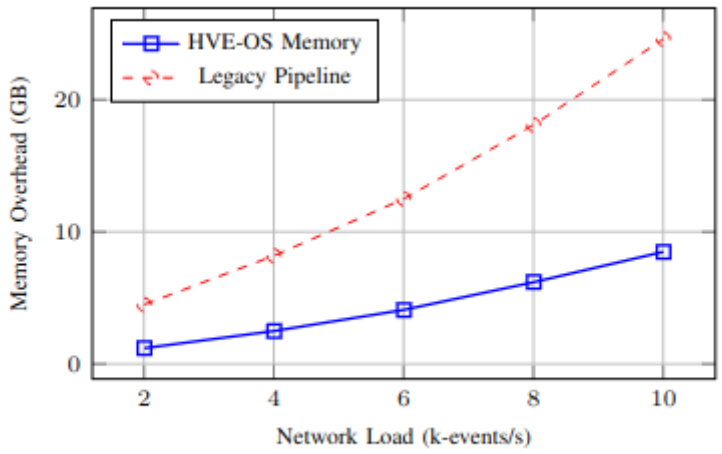

Fig. 2. Memory Efficiency: HVE-OS's modular execution reduces memory pressure significantly compared to monolithic pipelines [1], [4].

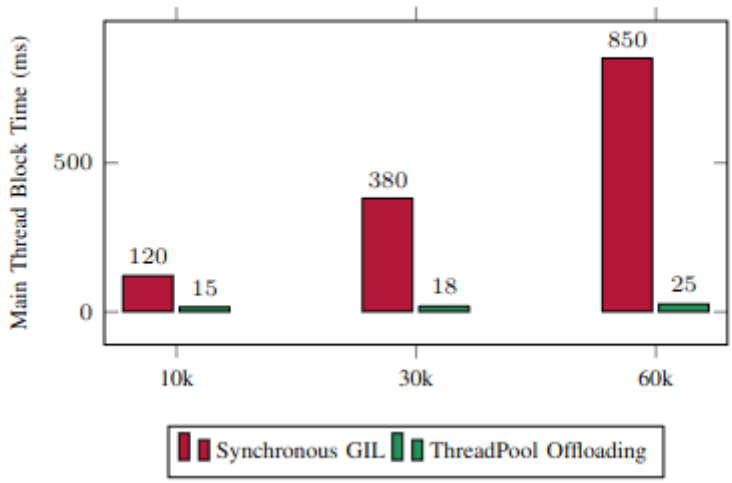

Figure 13: Bar charts showing memory efficiency and main-thread responsiveness under increasing workload, validating the thread-pool offloading design decision.

Code and Repository Reference

The project source code is available at. Important commands from the README include:
cd backend
docker-compose up -d
uvicorn app.main:app --reload --port 8000
cd frontend
npm run dev -- --force

Bibliography

1. David M. Herold and Lukasz Marzantowicz, "Supply chain responses to global disruptions and its ripple effects: an institutional complexity perspective," 2023.
2. Pervaiz Akhtar, Arsalan Mujahid Ghouri, Haseeb Ur Rehman Khan, Mirza Amin ul Haq, Usama Awan, Nadia Zahoor, Zaheer Khan, and Aniqa Ashraf, "Detecting fake news and disinformation using artificial intelligence and machine learning to avoid supply chain disruptions," 2023.
3. Maryam Shahsavari, Omar Khadeer Hussain, Morteza Saberi, and Pankaj Sharma, "Event Identification for Supply Chain Risk Management Through News Analysis by Using Large Language Models," 2024.
4. Kiran Katoor Vishnuthilak, Benjamin Rolf, Tobias Reggelin, and Sebastian Lang, "Using Sentiment Analysis to Detect Disruptive Events in Supply Chains," 2024.
5. Giuseppe Aiello, Cinzia Muriana, Salvatore Quaranta, and Islam Asem Salah Abushoyon, "A sustainable inventory management model for closed loop supply chain involving waste reduction and treatment," 2025.
6. SCOUT GitHub Repository: `https://github.com/scout-project`.
7. FastAPI Documentation, official framework documentation for Python API services.
8. Neo4j Documentation, official graph database and Cypher query documentation.
9. Redis Documentation, official in-memory data store documentation.
10. Apache Arrow Documentation, official PyArrow and Parquet format documentation.
11. React Flow Documentation, official documentation for node-based frontend workflow interfaces.

11 QR Code of Demonstration Video

Figure 14: QR Code of Demonstration Video